Know Your Images: what changed in your copy

Simone Bonato · 28 October 2026

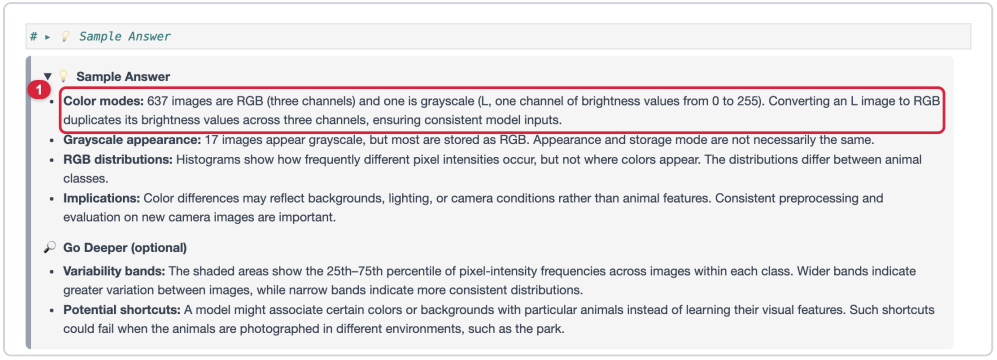
Thank you for the edited notebook. Your copy is now the notebook: nothing was merged back from the old version, and everything you removed stays removed. This document shows the biggest changes made on top of it, each with your version on the left and the new one on the right; the red numbers mark the exact place.

Changed under the hood, not shown: the hidden helper code was cleaned up (unused and repeated helpers removed, imports in one place); result variables are lowercase, settings in CAPITALS (PEP 8); the twelve image browsers no longer interfere with each other (clicking an earlier browser after running later chapters crashed or drew into the wrong cell); all Sample Answers use your gray box through one small helper; spelling is US throughout, "IOU" is "IoU"; references to section numbers point to your chapter names. Every cell runs on a free Colab CPU (tested headless, both notebooks).

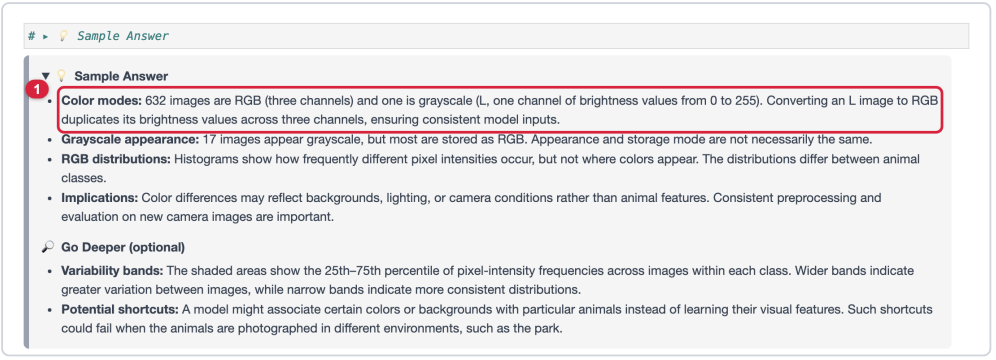
Answers now match the data

The Play Dataset has 633 images after the five photos were excluded on 26 October. Four Sample Answers still quoted 637 or 638 images (and 244 crowded images, "130 each"). They now quote what the notebook prints: 632 RGB, 241 of 633, 130 and 129.

Your copy (28 Oct)



New



The dog slider starts where your text is

The Class Imbalance task and its answer describe 20 dogs in 1,000 images (2 %, 98 % accuracy), but the slider started at 10 % and printed 90 %. It now starts at 2 %; students can still move it.

Your copy (28 Oct)

```
# ▸ 🔎 Explore Accuracy and Dog Recall Under Class Imbalance
```

```
Rule A — COCO subset (always 'cat')
  Accuracy:   20.5%
  Dog recall: 0.0%

Rule B — Paddock camera (10% dog images, always 'no dog')
  Accuracy:   90.0%
  Dog recall: 0.0%
```

New

```
# ▸ 🔎 Explore Accuracy and Dog Recall Under Class Imbalance
```

```
Rule A — COCO subset (always 'cat')
  Accuracy:   20.5%
  Dog recall: 0.0%

Rule B — Paddock camera (2% dog images, always 'no dog')
  Accuracy:   98.0%
  Dog recall: 0.0%
```

Image embeddings: fast on a classroom CPU

Your DINOv2 comparison is unchanged, and the model is the same. Your copy ran on an A100. On Colab's free CPU, installing the packages and embedding every image takes minutes, so the park's DINOv2 vectors are now computed once and loaded (they give your within-class similarities to three decimals). The model still runs on a GPU, or when COMPUTE_EMBEDDINGS is ticked.

Your copy (28 Oct)

```
# ▸ Helpers: Pretrained Visual Embeddings
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downlo
ads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable
higher rate limits and faster downloads.
preprocessor_config.json: 100% 436/436 [00:00<00:00, 53.7kB/s]
config.json: 100% 547/547 [00:00<00:00, 72.9kB/s]
model.safetensors: reconstructing file: 100% 88.2MB / 88.2MB, 8.47MB/s
model.safetensors: downloading bytes: 83.7MB, 7.91MB/s
Loading weights: 100% 223/223 [00:00<00:00, 3961.82it/s]
# ▸ Precompute Image Representations and UMAP
Extracting visual features: 100% 40/40 [00:06<00:00, 9.05it/s]
3 Computed and cached global_dinov2.
Computed and cached UMAP: Color histograms
Computed and cached UMAP: Pretrained visual embeddings
Ready: Color histograms, Pretrained visual embeddings
```

New

```
# ▸ Helpers: Pretrained Visual Embeddings
# ▸ Precompute Image Representations and UMAP
3 Pretrained visual embeddings: loaded from coco_animals_dinov2.npz (633 images, 384 numbers each), computed beforehand with facebook/d
inov2-small.
Loaded UMAP: Color histograms
Loaded UMAP: Pretrained visual embeddings
Ready: Color histograms, Pretrained visual embeddings
```

The end of the class notebook

Your marker and the old chapter 10 are gone from the class notebook: chapter 10 stopped Run all with an error (it used a cleaning step that your copy no longer has).
Your checklist and resources stay; a short closing points to the new Own-Data notebook and lists the guiding questions.

Your copy (28 Oct)

4 Continue here (SUS)

10. Your project dataset

```
# ▸ Checklist Dataset Mini-Project
```

Theory · What makes a good dataset

- **Does it fit the question?** The images, the classes and the setting must match the job you want the model to do.
  Park example: the alert needs a dog seen from a fixed camera in the sheep paddock, not a dog in a holiday snapshot taken from two metres away.
- **Enough samples, 2–10 classes** and **about 500×500 px**: the numbers to check, explained below.
- **Classes, distributions, imbalance, bias.** Know them, and take them into account in training (for example with a weighted loss).
  Park example: if 70 % of the photos are sheep and 3 % are dogs, the model sees few dogs to learn from, and the dog is the animal the alert exists for (section 4).
- **Variability matches the target population**, and **labels are present and validated**: every image has a label, and someone checked a sample of them.
  Park example: a photo of a zebra labelled "horse" teaches the model the wrong thing, and nobody notices until the zebra headcount is off.
- **Objects are identifiable and separable by eye.**
- **What do experts and the literature say** about this data and this task?
- **Get a qualitative and a quantitative sense of the data** (Karpathy, 2019): look at many images, then measure.

▸ **The reasons and park examples behind each point**

Your copy (28 Oct)

```
# ▸ Read your folder (or build the sample), and show what was found
```

```
5 NameError                                 Traceback (most recent call last)
/tmp/ipykernel_10835/3165945493.py in <cell line: 0>()
     28     my_source, my_root = "yours", find_dataset(MY_DATASET_DIR)
     29 else:
---> 30     my_source, my_root = "sample", build_my_sample(MY_SAMPLE_DIR, MY_SAMPLE_PER_CLASS, SEED)
     31     print("MY_DATASET_DIR is None: running on the park sample. Paste your path above to run on yours.")
     32 my_files = list_folder_per_class(my_root)

/tmp/ipykernel_10835/3165945493.py in build_my_sample(dest, per_class, seed)
      8     shutil.rmtree(dest, ignore_errors=True)  # rebuilt from scratch: re-running gives the same sample (N-013)
      9     rng = random.Random(seed)
---> 10     pool = clean_images[clean_images["flags"] == ""]
     11     picked = {}
     12     for c in CLASSES:

NameError: name 'clean_images' is not defined
```

New

Your Project Dataset

```
# ▸ Checklist Dataset Mini-Project
```

Theory · What makes a good dataset

- **Does it fit the question?** The images, the classes and the setting must match the job you want the model to do.
  Park example: the alert needs a dog seen from a fixed camera in the sheep paddock, not a dog in a holiday snapshot taken from two meters away.
- **Enough samples, 2–10 classes** and **about 500×500 px**: the numbers to check, explained below.
- **Classes, distributions, imbalance, bias.** Know them, and take them into account in training (for example with a weighted loss).
  Park example: if 70 % of the photos are sheep and 3 % are dogs, the model sees few dogs to learn from, and the dog is the animal the alert exists for (Class Imbalance in Meet the Park).
- **Variability matches the target population**, and **labels are present and validated**: every image has a label, and someone checked a sample of them.
  Park example: a photo of a zebra labeled "horse" teaches the model the wrong thing, and nobody notices until the zebra headcount is off.
- **Objects are identifiable and separable by eye.**
- **What do experts and the literature say** about this data and this task?
- **Get a qualitative and a quantitative sense of the data** (Karpathy, 2019): look at many images, then measure.

▸ **The reasons and park examples behind each point**

```
# ▸ 🔗 Resources: Image Datasets and Annotation Tools
```

🔗 **Resources: Image Datasets and Annotation Tools**

Where to find existing image datasets and tools for annotating your own images.

Find a dataset

- Kaggle datasets: a huge community collection, good for a first look.
- Google Dataset Search: one search box over many repositories.
- V7 open datasets: a curated list of open computer-vision data (plus their blog with overviews of well-known datasets).
- Roboflow Universe: community datasets, many already labeled with boxes or masks.
- LILA: wildlife and camera-trap images, the closest match to the park's cameras.
- torchvision datasets: classic datasets available directly through PyTorch.
- COCO: a general-purpose dataset; our Play Dataset is a subset of it.

Label your own images

- Supervisely: a labeling platform for boxes, polygons, and masks.
- V7: browser-based image annotation tools.
- Roboflow Annotate: browser-based labeling with export to common formats.
- Label Studio: open-source annotation software that can run locally.

To run these checks on your own project dataset, ope[4]e Own-Data Edition of this notebook, and use these questions to guide your exploration:

1. What is the task, and which mistake is the expensive one?
2. Do the images look like what the model will see in use?
3. How balanced are the classes, and is the expensive class a small one?
4. Which sizes, aspect ratios and color modes are there, and what must be resized or converted?
5. Are there copies, flipped or rotated copies, unreadable or odd files, or doubtful labels?
6. What ties photos together, and which split keeps those groups apart?
7. What must be fixed before training?

New: the Own-Data notebook

Built from the same cells as your notebook, so it never drifts from it. Students load their own dataset first; every analysis after that runs on their images. No Sample Answers: each task ends with a line for their answer and a one-line hint from the park.

Own-Data notebook: the opening

Know Your Images

6 **Exploring and Preparing Your Own Image Dataset**

Simone Bonato & Susanne Suter, FHNW

This notebook is the companion of the class notebook *Know Your Images*. In class you explored the images of the Wildwood Farm Park; here you run the same analyses on your own project dataset. You load your images and labels once, and every cell after that reads your data. There are no Sample Answers: each task ends with a line for your own answer, and a short *In the park* note reminds you what the same cell showed in class.

Your goal: Get to know your dataset well enough to explain what makes your task easy or difficult, and what must be fixed before training.

Inspired by Andrej Karpathy's A Recipe for Training Neural Networks, Step 1: Become one with the data.

A note about this notebook

This notebook was developed with AI assistance and subsequently reviewed, edited, and extended by the authors. While we have made every effort to ensure its accuracy, not every detail could be thoroughly verified.

If you encounter errors, unexpected behavior, or anything that seems incorrect or confusing, please let us know. Your feedback is especially valuable as this notebook is being used for the first time.

```
# ▸ Learning Objectives
```

Learning Objectives: Know Your Images

By the end of this notebook, you will be able to:

1. Explore and analyze image datasets qualitatively and quantitatively, and assess their suitability for a given task.
2. Identify and assess common dataset issues, including duplicates, incorrect labels, and invalid annotations.
3. Select and justify appropriate data splits and evaluation metrics, considering class imbalance, object size, and data leakage across computer vision tasks.

Activities are marked 🟢 core · 🔴 optional

What Do You Need to Know for the Exam and Your Project?

Focus on understanding the key concepts and applying them to new datasets. You should be able to explore and analyze your own image dataset, identify relevant characteristics and potential problems, interpret observations, and justify decisions about data preparation and evaluation.

You do **not** need to memorize specific dataset statistics, thresholds, code implementations, or individual examples. The activities help you develop the competencies needed for both the exam and your own project.

```
# ▸ How to Use This Notebook
```

How to Use This Notebook

- This is the Own-Data Edition. It runs the activities from class on your own dataset.
- Load your dataset first, in the chapter **Load Your Dataset**. Every later cell reads it.
- The Object Detection chapter runs only if your labels have boxes, and the Semantic Segmentation chapter only if they have masks.
- Write your answer below each task. The *In the park* note tells you what to look for.
- The notebook runs in Google Colab. A CPU is sufficient for most activities, although computing image embeddings can take longer without a GPU.

How students learn the label formats

One page explains what each task needs and how each format stores a box or a mask, using the same sheep box in every format so students can compare the numbers.

Own-Data notebook: Load Your Dataset

Load Your Dataset

In class you explored the Wildwood Farm Park. This notebook runs the same checks on **your** dataset: every cell after this chapter reads your images and labels instead of the park's. Where it helps, a short *In the park* note reminds you what the same cell showed there, so you know what to look for.

> ▶️ **Run the cells of this chapter in order.** When your dataset is loaded, use *Runtime → Run all* or go chapter by chapter. Chapters your labels cannot support (for example Object Detection without boxes) skip themselves with one line saying why.

```
# ▸ 📘 What Your Dataset Must Contain
```

> 📘 **What Your Dataset Must Contain**
>
> Each computer vision task needs a different kind of label:
>
> - **Classification:** one class per image. *"This image shows a sheep."*
> - **Object detection:** a **box** around every object, with its class. *"There is a sheep inside this rectangle."*
> - **Semantic segmentation:** a **mask** of every object, with its class: the exact pixels it covers.
>
> Labels build on each other: from a mask we compute its box, and from the boxes in an image we take the most frequent class as the image's class. So a dataset with masks can run all three chapters, a dataset with boxes the first two, and a dataset with class labels only the classification chapters.

How Each Format Stores Its Labels

You do not convert anything yourself. Tell the notebook which tasks your labels support, and it reads any of the formats below and converts them internally. If your labeling tool offers several exports, **COCO JSON** is the safest choice: it holds boxes and masks in one file.

To compare the formats, every example below describes **the same box**: a sheep in a 640 × 480 image, whose box starts 120 pixels from the left edge and 80 from the top, and is 200 pixels wide and 150 pixels high.

Classification

- **One folder per class** (`"folders"`): the folder name is the class.

```
my_dataset/
├── horse/   img_001.jpg, img_002.jpg, …
└── sheep/   img_101.jpg, …
```

- **A CSV file** (`"csv"`): one row per image, with a `file` column (the path from the CSV's folder) and a `label` column.

```
file,label
images/img_101.jpg,sheep
```

Object detection (boxes). A box is always four numbers. The formats differ in *which* four:

Format	One file per	The four numbers	Our sheep
COCO JSON (`"coco"`)	dataset	x, y of the top-left corner, width, height, in pixels	`[120, 80, 200, 150]`
YOLO (`"yolo"`)	image (`.txt`)	x, y of the **center**, width, height, as **fractions** of the image size (0 to 1)	`0.3438 0.3229 0.3125 0.3125`
Pascal VOC (`"voc"`)	image (`.xml`)	x, y of the top-left corner, x, y of the **bottom-right** corner, in pixels	`120 80 320 230`

- **COCO JSON:** one file lists the images, the annotations and the class names. Each annotation points to its image (`image_id`) and its class (`category_id`):

```
{"images":      [{"id": 1, "file_name": "img_101.jpg", "width": 640, "height": 480}],
 "annotations": [{"id": 1, "image_id": 1, "category_id": 2, "bbox": [120, 80, 200, 150]}],
 "categories":  [{"id": 2, "name": "sheep"}]}
```

- **YOLO:** `labels/img_101.txt` has one line per object, starting with the class **index**; the class names are in `data.yaml` (`names: [horse, sheep]` , so index 1 is sheep). An image with no objects has an empty or no `.txt` file.

```
1 0.3438 0.3229 0.3125 0.3125
```

- **Pascal VOC:** `labels/img_101.xml` has one `<object>` per object:

```
<object><name>sheep</name>
  <bndbox><xmin>120</xmin><ymin>80</ymin><xmax>320</xmax><ymax>230</ymax></bndbox></object>
```

Loading a dataset, in any common format

Students set three things: where the data is, which tasks their labels support (classification, + boxes, + masks), and the format (found automatically). Folders, CSV, COCO, YOLO, Pascal VOC and PNG masks are converted to the same tables your chapters use. The notebook then shows one image with its labels exactly as stored in their files.

Settings, the load summary and one labeled image

```
# ✏️ YOUR TURN
DATASET_PATH = None  # your folder, .zip, or a Google Drive link to a .zip; None = the park sample
TASKS = ["classification", "detection", "segmentation"]
LABEL_FORMAT = "coco"  # @param ["auto", "folders", "csv", "coco", "yolo", "voc", "png-masks"]
```

```
# ▸ Helpers: Find and Open Your Dataset
```

```
# ▸ Helpers: Read Your Labels (six formats, one table)
```

```
# ▸ Helpers: Park Sample, Class Colors and Skipped Cells
```

```
# ▸ Load Your Dataset (or the Park Sample)
```

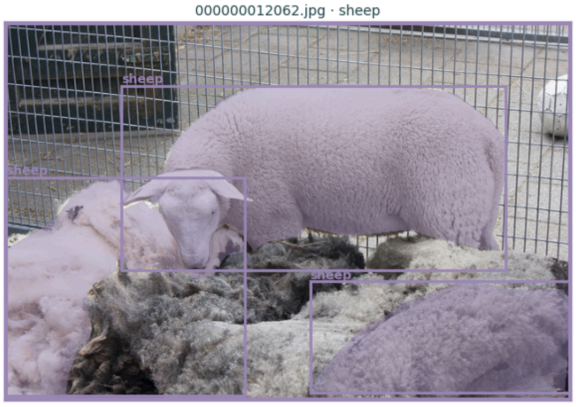
```
120 images to fetch from images.cocodataset.org (cached in data/cache/coco/)
  images ready: 120/120
✅ source: the park sample, written as coco (DATASET_PATH is None) · format: coco · 120 images · 388 objects (boxes)
6 classes: bird 20 · cat 20 · dog 20 · horse 20 · sheep 20 · zebra 20
tasks that will run: classification, detection, segmentation
```

```
# ▸ 🔎 See How One Image Is Labeled
```

```
000000012062.jpg · class: sheep · 640 × 427 px

How your files (coco) store its labels:
    images/annotations.json, the image:
    {"id": 12062, "file_name": "000000012062.jpg", "width": 640, "height": 427}
    its 3 annotation(s):
    {"id": 64311, "image_id": 12062, "category_id": 5, "bbox": [130.68, 70.4, 435.28, 208.9], "area": 62556.14115000001, "segmentatio
n": [[178.53, 169.79, 193.25, 132.06, 242.95, 87.88, 277.92, 74.08, "… 74 more numbers"]], "iscrowd": 0}
    {"id": 64787, "image_id": 12062, "category_id": 5, "bbox": [0.0, 174.53, 270.89, 247.94], "area": 27289.100100000007, "segmentati
on": [[0.0, 257.18, 44.08, 230.55, 59.69, 218.61, 74.38, 198.41, "… 88 more numbers"]], "iscrowd": 0}
    {"id": 1405357, "image_id": 12062, "category_id": 5, "bbox": [344.48, 292.66, 295.52, 125.7], "area": 27975.93394999999, "segment
ation": [[640.0, 314.73, 632.34, 302.26, 579.57, 303.22, 520.08, 292.66, "… 12 more numbers"]], "iscrowd": 0}
    their categories:
    {"id": 5, "name": "sheep"}

The same boxes in the notebook's `annotations` table (x, y of the top-left corner, width, height, in pixels):
    sheep: [131, 70, 435, 209]
    sheep: [0, 175, 271, 248]
    sheep: [344, 293, 296, 126]
```



Same task, two notebooks

Left: your task in the class notebook, with its Sample Answer. Right: the same task in the Own-Data notebook, with an answer line and your answer turned into a one-line hint.

Your copy (28 Oct)

`# ▸ 💡 Sample Answer`

▸ 💡 **Sample Answer**

🟢 Image Dimensions: What Do Their Distributions Reveal?

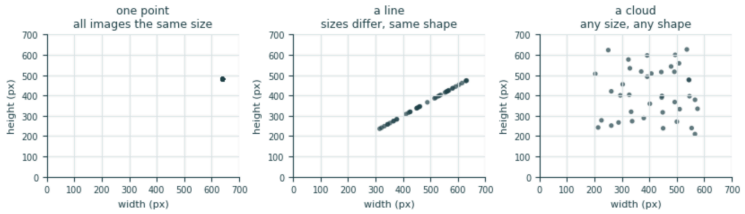
We've examined individual images and summary statistics. Now let's visualize how image dimensions and aspect ratios vary across the dataset.

The COCO images come from many different photographers, while the park will use a few fixed cameras.

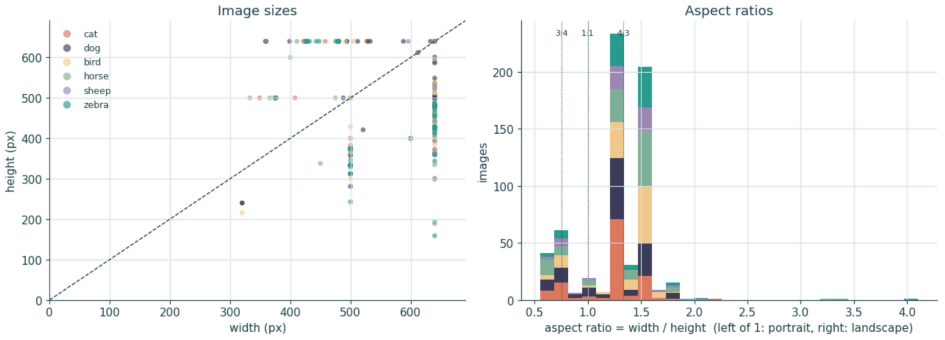
Observe:

- What patterns do you see in the width-versus-height scatter plot?
- How do aspect ratios differ between animal classes?
- What might these differences mean when preparing images for a computer vision model?

`# ▸ 🔎 Understanding Width–Height Scatter Plot Patterns`



`# ▸ 🔎 Explore Image Dimensions and Aspect Ratios by Class`



10 `▸ 💡 Sample Answer`

▸ 💡 **Sample Answer**

Own-Data notebook

🟢 Image Dimensions: What Do Their Distributions Reveal?

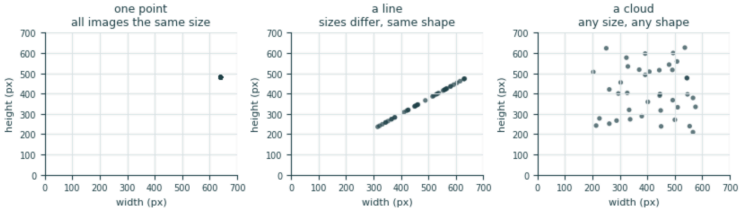
We've examined individual images and summary statistics. Now let's visualize how image dimensions and aspect ratios vary across your dataset.

Images from one camera tend to share one size. Images collected from many sources come in many sizes.

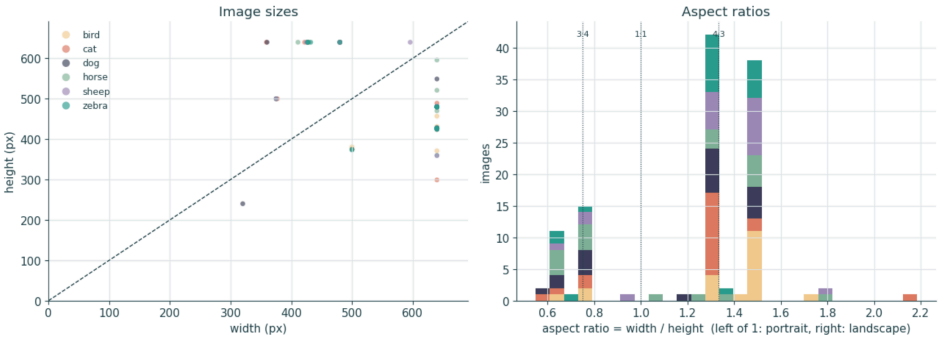
Observe:

- What patterns do you see in the width-versus-height scatter plot?
- How do aspect ratios differ between classes?
- What might these differences mean when preparing images for a computer vision model?

`# ▸ 🔎 Understanding Width–Height Scatter Plot Patterns`



`# ▸ 🔎 Explore Image Dimensions and Aspect Ratios by Class`



10 **Your answer** (double-click to edit):

11 *In the park:* the scatter plot showed horizontal and vertical bands around 500 and 640 pixels, a sign of standardized sizes, with common ratios around 4:3 and 3:4.

Chapters the data cannot support skip themselves

A classification-only dataset has no boxes, so every Object Detection cell says it was skipped instead of failing. Run all always reaches the end.

Own-Data notebook, classification only

Object Detection

In image classification, a model predicts what is present in an image. But what if you need to count individual objects and know where they are?

This chapter runs only if `TASKS` includes `"detection"`, so its cells skip themselves when your labels have no boxes.

Object detection extends classification by predicting a class label and a bounding box for each object. This allows us to investigate not only which objects appear, but also how many there are, where they are located, and how large they appear.

In this section, we explore how object size, position, crowding, and class imbalance affect your object detection dataset. We also examine the additional annotation effort required compared with image classification.

From Labels to Bounding Boxes

Object detection datasets contain one annotation per object, not one label per image. Each annotation specifies the object's class and bounding box.

When you loaded your dataset, the notebook converted your boxes to the COCO format `[x, y, w, h]`, whatever format you stored them in. Here `x` and `y` indicate the top-left corner and `w` and `h` specify the width and height in pixels.

Unlike image classification, an image can contain multiple annotated objects, potentially belonging to different classes.

We explore four characteristics of the object detection dataset:

- **Class imbalance:** How are object instances distributed across classes?
- **Object size:** How large or small are the annotated objects?
- **Object position:** Where are objects located within images?
- **Crowding:** How many objects appear together, and how much do they overlap?

🟢 Exploring Bounding Boxes: What Do You Notice?

Before running the code:

- How might the number and size of annotated objects differ between images?

Explore:

- Examine the four example images and their bounding boxes.

Observe and interpret:

- How many objects are annotated in each image?
- How do the object sizes vary, and why might this matter for object detection?

```
# ▸ Helpers: Bounding Boxes
```

```
▸ 🔎 Explore Bounding Boxes
```

12

```
Skipped: needs boxes.
```

🟢 Boxes per Class vs. Photos per Class

In image classification, class distributions are typically counted by images. In object detection, we can also count individual objects. For example, one image containing 12 objects contributes one image but 12 bounding boxes.

Before running the code:

- Do you expect the class ranking to change when counting bounding boxes instead of images? Which class might increase the most?

Coding Exercise:

In this and subsequent coding exercises, you will implement small functions to explore the dataset. Complete the provided code cells and check your results. If you get stuck, you can reveal the reference solution. The notebook continues to work even if your implementation is incomplete.

- Implement `count_boxes_per_class(annotations)` to return the number of bounding boxes per class.
- Run the validation and compare the resulting distribution with the image-level distribution from Class Distribution: Are All Classes Equally Represented?

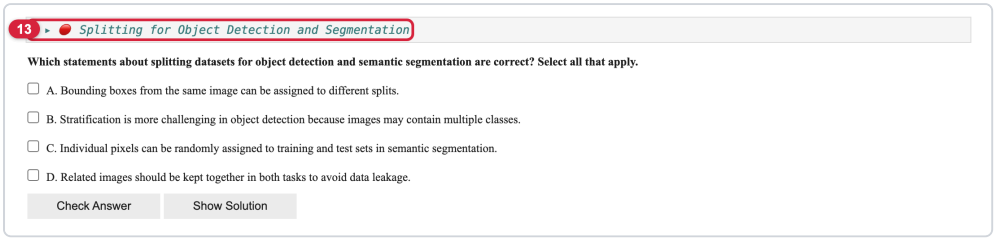
Observe and interpret:

- Which classes change most in their relative frequency?
- Why might image-level and object-level class imbalance lead to different conclusions?

Your multiple-choice question is in the table of contents

It had its level dot only in a code-cell title, so Colab's table of contents did not list it. It now has a heading like your other tasks; every 🔴 heading carries "(Optional)".

Your copy (28 Oct)



New

